EECS 31L: INTRODUCTION TO DIGITAL DESIGN LAB
LECTURE 5

Salma Elmalaki
salma.elmalaki@uci.edu

The Henry Samueli School of Engineering
Electrical Engineering and Computer Science
University of California, Irvine

Some slides are compiled from JL Gaudiot and P. Yaghini

LECTURE 5: LAST TIME

- `Generate for`: concurrent execution
- Behavioral Description
- Sequential statements
  - if, case, …

LECTURE 5: OVERVIEW

- Case, For and While statements
- Storage elements
  - DFF
  - JK FF
  - Binary counters - Self read examples in the book
- Blocking vs. non blocking assignment
- Structural description
- Binding and port instantiation
- Prepare for the online midterm (Midterm is on Feb 18 respondus lockdown browser with webcam)

CASE STATEMENTS

- **Case** statement:
  - The case statement selects for execution one of several alternative sequences of statements the alternative is chosen based on the value of associated expression.
  - Must cover all possible options
- Simple but rigid!
- It uses the verify operator === to do bitwise comparison
- Everything must be explicitly coded

```
case (control-expression)
 value : begin statement1; end
 value : begin statement2; end
 default:begin default statement; end
 endcase
```

```
 case (sel)
  2'b00 : begin y = I1; z=I1&I2; end
  2'b01 : y = I2;
  2'b10 : y = I3;
  default: y = I4;
 endcase
```

Single statement does not need begin/end

CASE, CASEX, CASEZ STATEMENTS

- **Which statements will be executed if a = 1'b0?**

```
case ( a )
  1'b0 : statement1;
  1'b1 : statement2;
  1'bx : statement3;
  1'bz : statement4;
endcase
```

```
casez ( a )
  1'b0 : statement1;
  1'b1 : statement2;
  1'bx : statement3;
  1'bz : statement4;
endcase
```

```
casex ( a )
  1'b0 : statement1;
  1'b1 : statement2;
  1'bx : statement3;
  1'bz : statement4;
endcase
```

- The **case** statement uses === for comparison
- The **casez** statement ignores high-impedance (z) bits.
- The **casex** statement ignores high-impedance (z) and unknown (x) bits.
- If any of the bits in the case expression or case item expression is an ignored value then that bit position will be ignored.

CASE, CASEX, CASEZ STATEMENTS

- **Which statements will be executed if a = 1'b1?**

```
case ( a )
  1'b0 : statement1;
  1'b1 : statement2;
  1'bx : statement3;
  1'bz : statement4;
endcase
```

```
casez ( a )
  1'b0 : statement1;
  1'b1 : statement2;
  1'bx : statement3;
  1'bz : statement4;
endcase
```

```
casex ( a )
  1'b0 : statement1;
  1'b1 : statement2;
  1'bx : statement3;
  1'bz : statement4;
endcase
```

- The **case** statement uses === for comparison
- The **casez** statement ignores high-impedance (z) bits.
- The **casex** statement ignores high-impedance (z) and unknown (x) bits.
- If any of the bits in the case expression or case item expression is an ignored value then that bit position will be ignored.

CASE, CASEX, CASEZ STATEMENTS

- **Which statements will be executed if a = 1'bz?** First Match Wins!

```
case ( a )
  1'b0 : statement1;
  1'b1 : statement2;
  1'bx : statement3;
  1'bz : statement4;
endcase
```

```
casez ( a )
  1'b0 : statement1;
  1'b1 : statement2;
  1'bx : statement3;
  1'bz : statement4;
endcase
```

```
casex ( a )
  1'b0 : statement1;
  1'b1 : statement2;
  1'bx : statement3;
  1'bz : statement4;
endcase
```

- The **case** statement uses === for comparison
- The **casez** statement ignores high-impedance (z) bits.
- The **casex** statement ignores high-impedance (z) and unknown (x) bits.
- If any of the bits in the case expression or case item expression is an ignored value then that bit position will be ignored.

CASE, CASEX, CASEZ STATEMENTS

- **Which statements will be executed if a = 1'bx?** First Match Wins!

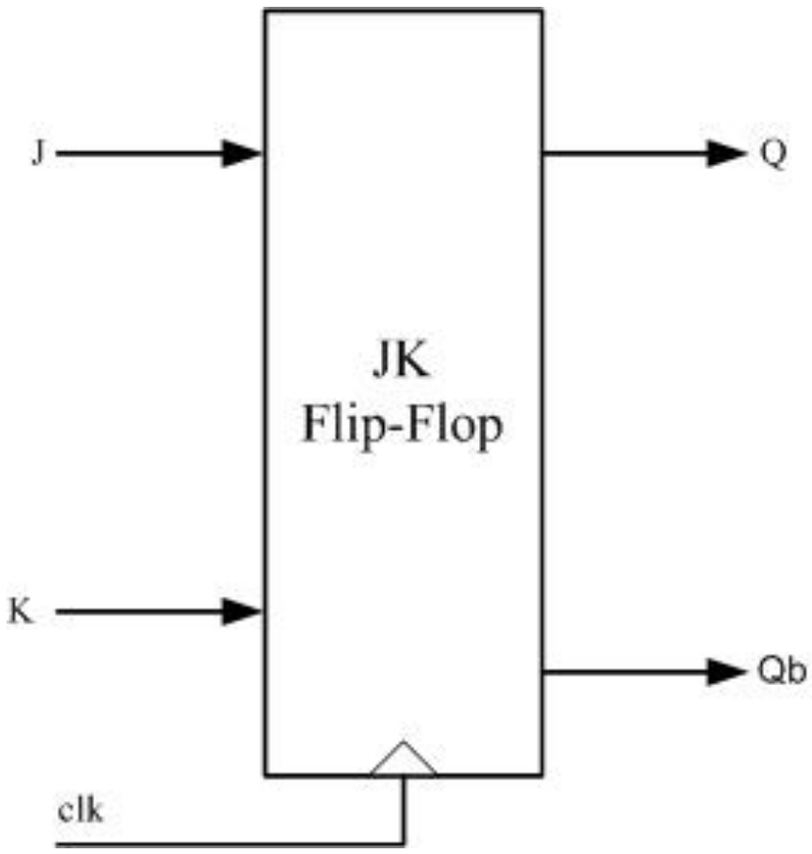
```
case ( a )
  1'b0 : statement1;
  1'b1 : statement2;
  1'bx : statement3;
  1'bz : statement4;
endcase
```

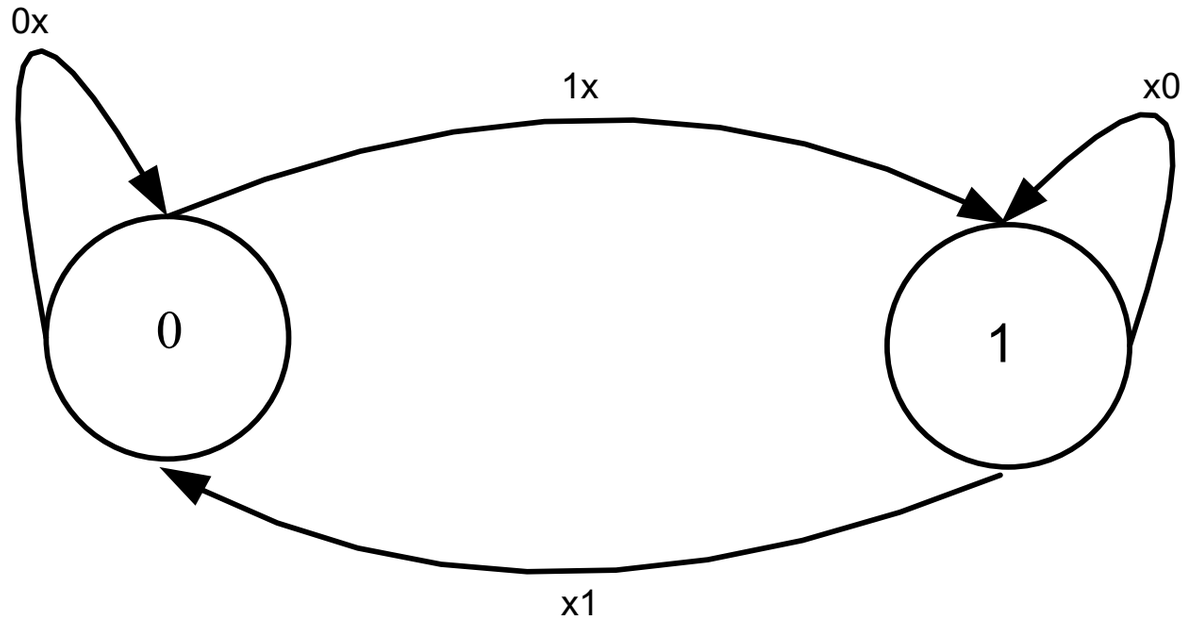
```
casez ( a )
  1'b0 : statement1;
  1'b1 : statement2;
  1'bx : statement3;
  1'bz : statement4;
endcase
```

```
casex ( a )
  1'b0 : statement1;
  1'b1 : statement2;
  1'bx : statement3;
  1'bz : statement4;
endcase
```

- The **case** statement uses === for comparison
- The **casez** statement ignores high-impedance (z) bits.
- The **casex** statement ignores high-impedance (z) and unknown (x) bits.
- If any of the bits in the case expression or case item expression is an ignored value then that bit position will be ignored.

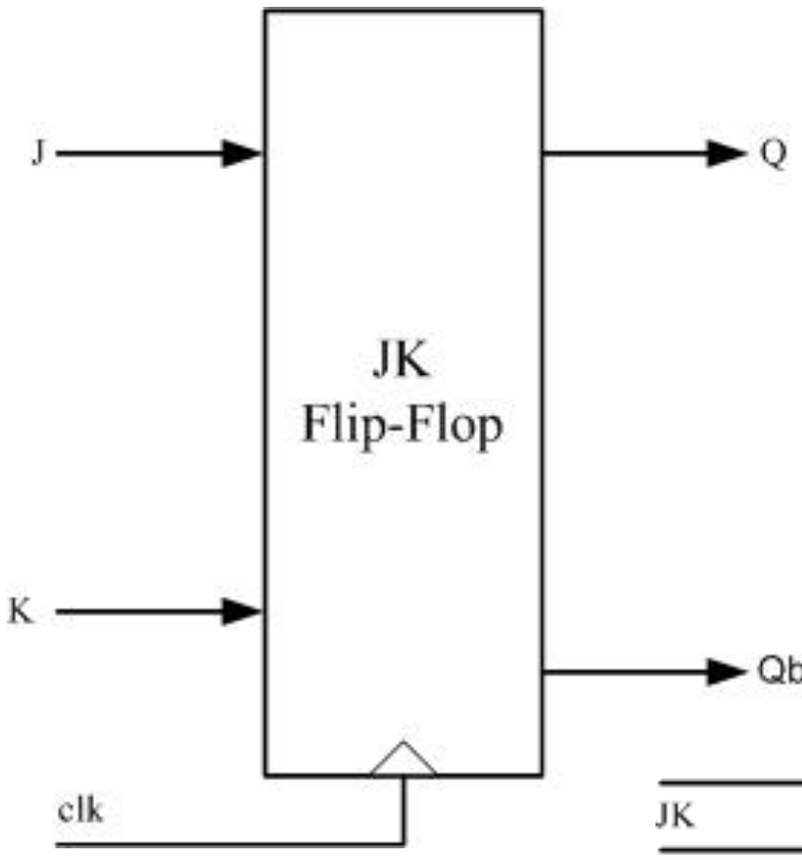
EXAMPLE ON CASE: POSITIVE EDGE TRIGGERED J-K FF

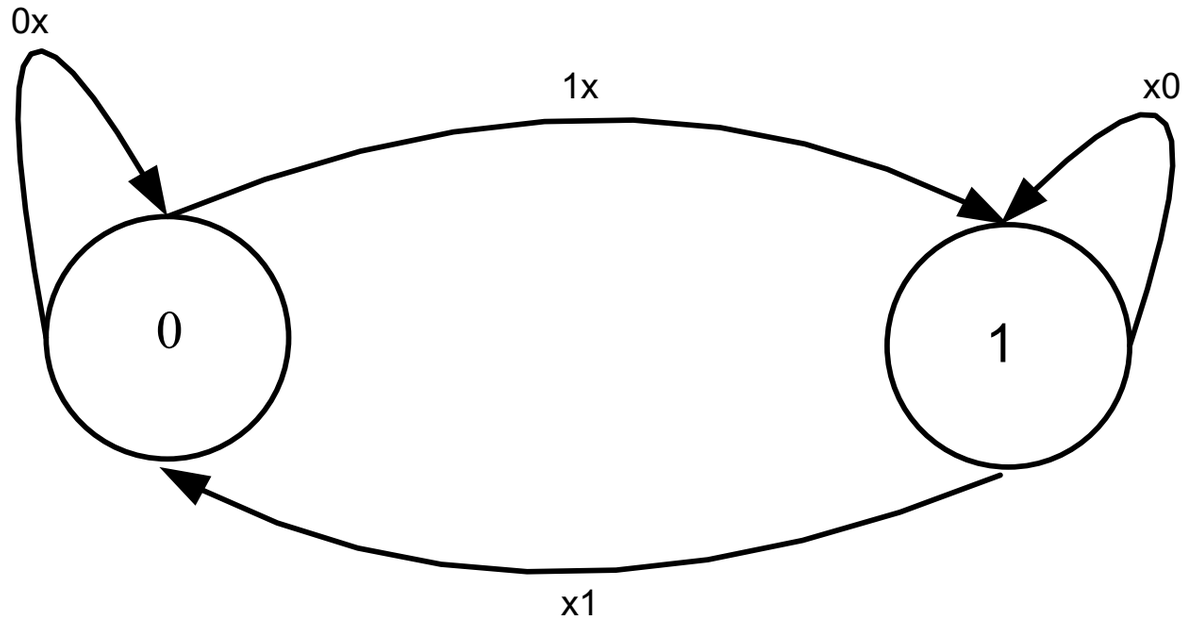


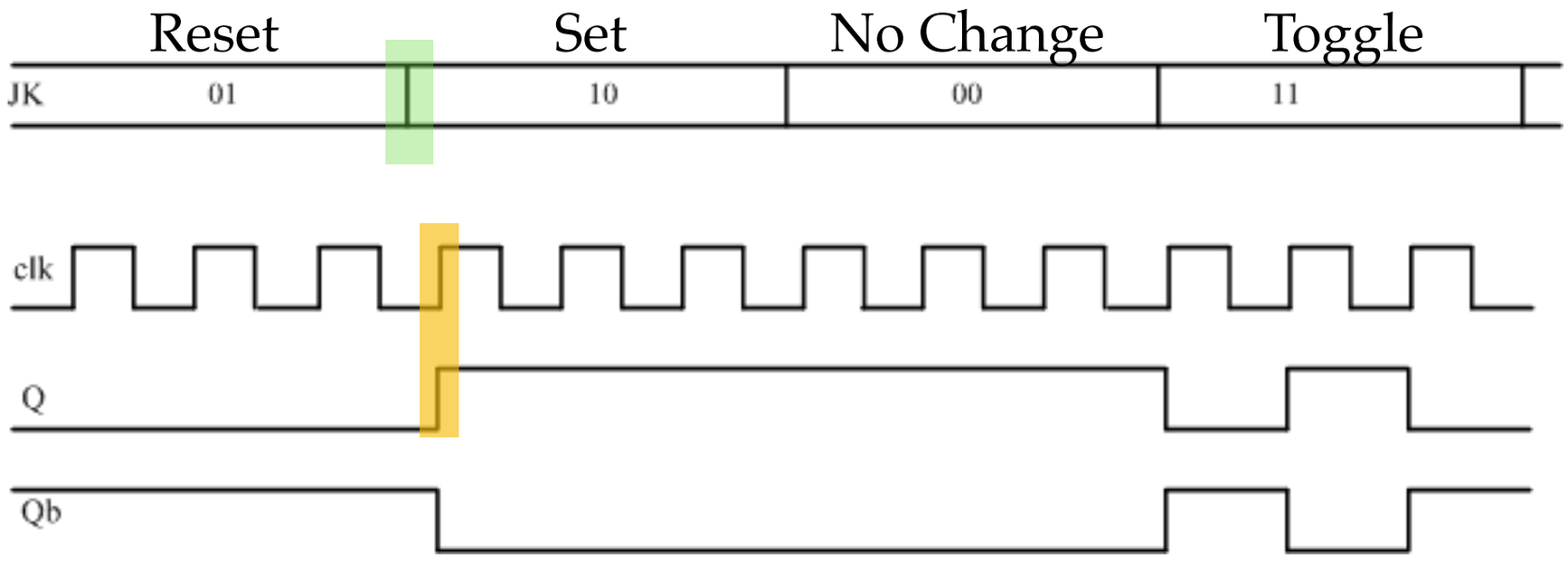


Clk	J	K	Q_{n+1}	Meaning
↑	0	0	Q_n	No change
↑	0	1	0	Reset
↑	1	0	1	Set
↑	1	1	$\bar{Q}_n$	Toggle

EXAMPLE ON CASE: POSITIVE EDGE TRIGGERED J-K FF





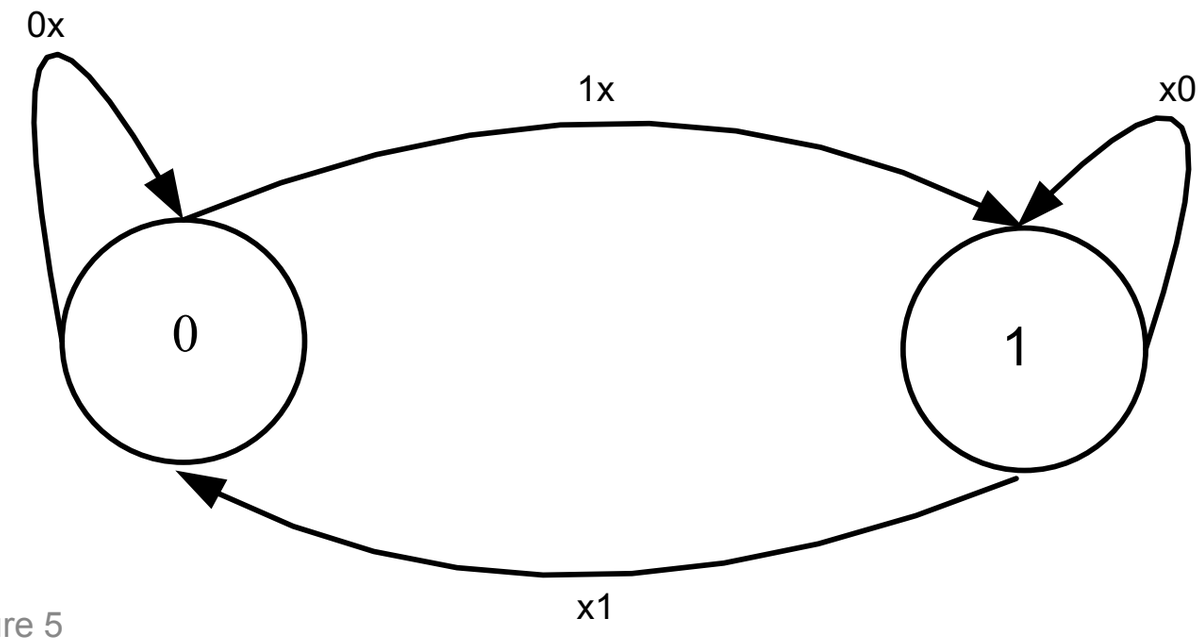


EXAMPLE ON CASE: POSITIVE EDGE TRIGGERED J-K FF

```
module JK_FF (JK,clk,q,q_bar);
input [1:0] JK;
input clk;
output reg q,q_bar;

always @(posedge clk)
begin
  case (JK)
     2'b00 : q=q;
     2'b01 : q=1'b0;
     2'b10 : q=1'b1;
     2'b11 : q=~q;
  endcsae
q_bar = ~q;
end
endmodule
```

posedge is a keyword meaning at the rising edge of the clock

Remember the parenthesis around the case condition



SEQUENTIAL STATEMENTS

- Sequential statements includes:
  - if
    - D-Latch example
  - case
    - JK-FF
  - wait
  - loop

WAIT STATEMENT

- Wait for a time period
- Can be used to generate clocks with different time periods

```
wait (condition) #(<optional_delay>) statement
```

wait (i>10&&j<5) #10 a=b;

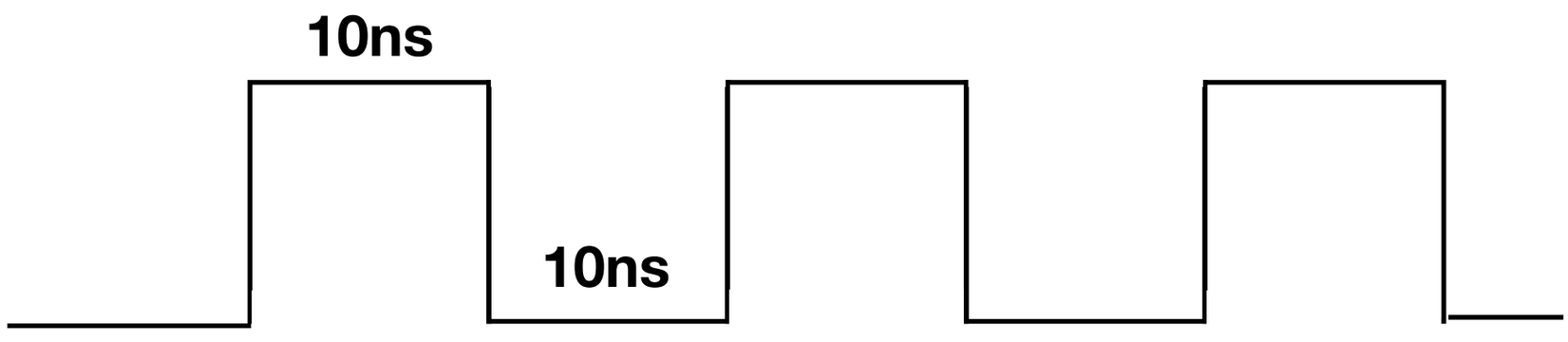
```
#delay
```

 # 10;

Component	Action
`wait (...)`	Pauses until the logical condition is true.
`#10`	Waits an additional 10 time units.
`a = b;`	Assigns the value of `b` to `a` immediately.

EXAMPLE ON WAIT: CLOCK GENERATION

```
module wait_statement (a,b,c)
output reg a,b,c;

initial
    begin
        a=0;
    end
//initial is done

always
    begin
    #10 ;
            a = ~ a;
    end

endmodule
```



Time period = 20 ns

SEQUENTIAL STATEMENTS

- Sequential statements includes:
  - if
    - D-Latch example
  - case
    - JK-FF
  - wait
    - Clock generation
  - loop
    - for
    - while
    - repeat
    - forever

LOOP STATEMENTS

- For loop

```
for(counter_low;counter_up;counter_update)
  begin
     sequential_statements;
  end
```

```
integer i;
for (i=0;i<=2;i=i+1)
begin
 if (temp[i] == 1'b1)
   begin
      result = result + 2**i;
   end
end
statement1;
statement2;
```

Don't forget to declare the index i

What does this loop do?

LOOP STATEMENTS

- While loop

```
while(condition)
  begin
     sequential_statements;
  end
```

```
integer i=100;
while (i>1)
  begin
     j= i+1;
     i= i/2;
  end
```

Don’t forget to declare the index i

LOOP STATEMENTS

- **Repeat** loop
  - No condition is allowed

```
repeat(fixed_number)
  begin
    sequential_statements;
  end
```

- **Forever** loop
  - No condition
  - Loop endlessly
  - Use for clock generation

```
initial
  begin
    clk=1'b0;
    forever #20clk=~clk;
  end
```

LOOP SUMMARY

Loop statements provide a means of modeling blocks of behavioral statements.

- **forever**

  continuously repeats the statement that follows it. Therefore, it should be used with timing controls (otherwise it hangs the simulation).

- **repeat**

  executes a given statement a fixed number of times. The number of executions is set by the expression, which follows the repeat keyword. If the expression evaluates to unknown, high-impedance, or a zero value, then no statement will be executed.

- **while**

  executes a given statement until the expression is true. If a while statement starts with a false value, then no statement will be executed.

- **for**

  executes a given statement until the expression is true. At the initial step, the first assignment will be executed. At the second step, the expression will be evaluated. If the expression evaluates to an unknown, high-impedance, or zero value, then the for statement will be terminated. Otherwise, the statement and second assignment will be executed. After that, the second step is repeated.

MORE EXAMPLES IN THE BOOK

- 4-bit counter with synchronous clear
- 4-bit counter with synchronous hold.
- Shift registers
- Calculating factorial

Break 5 minutes

SO FAR: BEHAVIORAL DESCRIPTION

- Basic statements of behavioral description
  - always/initial blocks
- Sequential statements
  - If
    - D-latch FF
  - Case
    - JK FF
  - Wait
    - clock generation
  - loop (for, while, repeat, forever)

- Blocking vs. non-blocking assignments

LECTURE 5: OVERVIEW

- Blocking vs. non blocking assignment
- Structural description
- Binding and port instantiation

BLOCKING VS. NON-BLOCKING ASSIGNMENT

Inside sequential block (like always or initial)

- **blocking**
  - Use **=** operator
  - The result of this assignment can be seen in the subsequent statements
  - Statements are executed sequentially instead of concurrently
- **non-blocking**
  - Use **<=** operator
  - The result of this assignment cannot be seen in subsequent statements
  - Statements are executed concurrently instead of sequentially
  - The order of non-blocking assignments is not important as long as the destinations of the assignments are different signals

BLOCKING VS NON-BLOCKING

Blocking assignment: evaluation and assignment are immediate

```
always @ (a or b or c)
begin
   x = a | b;          1. Evaluate a | b, assign result to x
   y = a ^ b ^ c;      2. Evaluate a^b^c, assign result to y
   z = b & ~c;         3. Evaluate b&(~c), assign result to z
end
```

Non-Blocking assignment: all assignments are deferred until all right-hand sides have been evaluated (end of simulation timestep)

```
always @ (a or b or c)
begin
   x <= a | b;         1. Evaluate a | b but defer assignment of x
   y <= a ^ b ^ c;     2. Evaluate a^b^c but defer assignment of y
   z <= b & ~c;        3. Evaluate b&(~c) but defer assignment of z
end                    4. Assign x, y, and z with their new values
```

- Sometimes, as above, both produce the same result. Sometimes, not!

Slide adopted from course 6.111 MIT

EXAMPLE; BLOCKING VS. NON-BLOCKING ASSIGNMENT

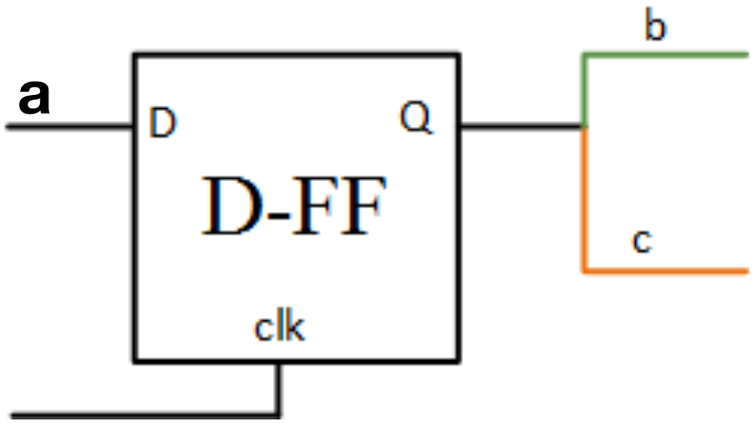
```
module block_nonblock();
reg a, b, c, d , e, f ;

 // Blocking assignments
initial begin
 #10 a = 1'b1;
 #20 b = 1'b0;
 #40 c = 1'b1;
end

/* The simulator assigns
1 to a at time 10, 0 to
b at 30, and 1 to c at
70
*/
```

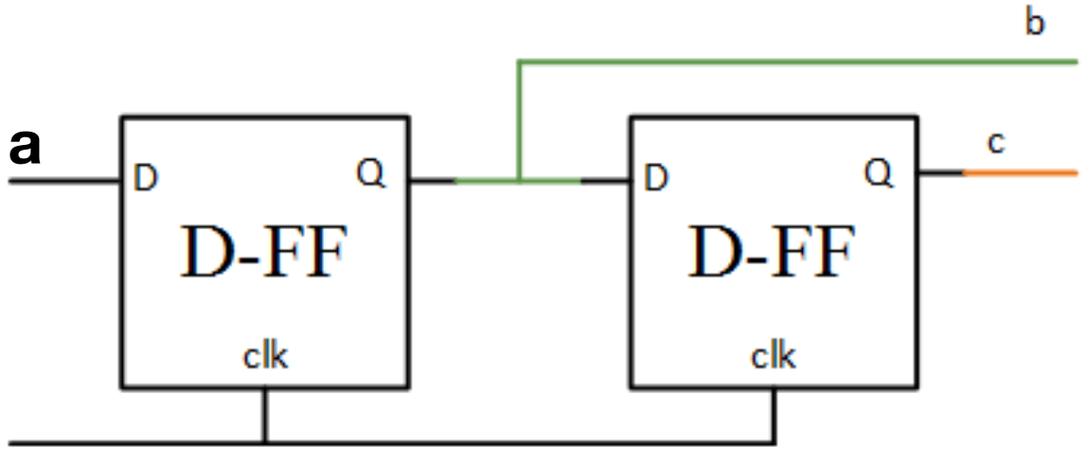
```
// Non-blocking
assignments
initial begin
 #10  d <=  1'b1;
 #20  e <=  1'b0;
 #40  f <=  1'b1;
end
endmodule

/* The simulator
assigns 1 to d at time
10, 0 to e at 20, and 1
to f at 40 */
```

EXAMPLE; BLOCKING VS. NON-BLOCKING ASSIGNMENT

```
module blocking (clk,a,c);
   input clk;
   input a;
   output c;
   wire clk,a;
   reg c,b;
always @ (posedge clk )
 begin
    b = a;
    c = b;
 end
endmodule
```

```
module non-blocking (clk,a,c);
   input clk;
   input a;
   output c;
   wire clk,a;
   reg c,b;
always @ (posedge clk )
 begin
    b <= a;
    c <= b;
 end
endmodule
```





STRUCTURAL DESCRIPTION

- Best option when
  - Hardware details of the design are known
  - Schematic solution is available
- Structural description simulates the system by describing its logical components.
  - Components can be gate level(such as AND gates, OR gates, or NOT gates)
  - Components can be in a higher logical level, such as Register- Transfer Level (RTL) or processor level.
- Verilog recognizes all the primitive gates, such as AND, OR, XOR, NOT, and XNOR gates.
- Basic VHDL packages do not recognize any gates unless the package is linked to one or more libraries.

STRUCTURAL DESCRIPTION

Verilog Structural Description

```
module system(a,b,sum,cout);

    input a,b;

    output sum, cout;

    xor X1(sum, a, b);  // (outputs, inputs)   /* X1 is an optional identifier; it can be omitted.*/

    and a1(cout, a, b);    /* a1 is optional identifier; it can be omitted.*/

endmodule
```

What if you want to build your primitive cell?

BINDING AND INSTANTIATION

```
module two (s1, s2, a1, b1);
    input a1;
    input  b1;
    output  s1, s2;
    xor (s1, a1, b1);
    and (s2, a1, b1);
endmodule
```

BINDING AND INSTANTIATION

Binding between two Modules in Verilog

```
module one( O1,O2,a,b);
     input [1:0] a;
     input [1:0] b;
     output [1:0] O1, O2;

two M0(O1[0], O2[0], a[0], b[0]);
two M1(O1[1], O2[1], a[1], b[1]);
endmodule

module two (s1, s2, a1, b1);
     input a1;
     input  b1;
     output  s1, s2;
     xor (s1, a1, b1);
     and (s2, a1, b1);
endmodule
```

Binds module two to module one
The relationship as follows:

- `O1[0]` is the output of a two-input XOR gate with `a[0]` and `b[0]` as the inputs.
- `O2[1]` is the output of a two-input AND gate with `a[1]` and `b[1]` as the inputs.

Binds module two to module one

- Port Association

Implicit by order (Position Mapping)

- Note: You cannot instantiate modules inside always blocks.